

Harness Engineering

Harness是什么？

翻译过来就是马具，AI是那匹马。 harness engineering就是怎么让这匹马跑得更快更稳
具体定义就是

Harness = 围绕AI模型搭建的一整套工作环境和 workflows

项目规则文件+配套的各种工具+任务拆分和执行顺序+测试检查流程->这些都算是Harness

AI工程的演示过程：

2022：提示词工程（教AI听懂需求）

怎么通过对话让AI听懂你的需求

设定角色->约束格式->思维链->示例模仿

示例：你是一个资深的后端工程师，请一步步思考这个问题1->2->3，参考这个示例回答，用JSON格式输出

2024：上下文工程（给AI喂对信息）

怎么在对的时候把对的信息喂给AI

AGENT.MD规则->RAG检索->上下文压缩->跨对话记忆

示例；AGENTS.MD中了解项目背景，用RAG让能检索到相关资料，对过长的上下文作压缩和摘要，甚至给AI建立跨对话的记忆机制

2026：Harness Engineering(让ai靠谱地干完整件事)

让AI持续靠谱地完成整个任务

配工具->拆任务->自检修复->防止质量下滑

Agent = 模型 + harness

1.上下文架构：让AI了解背景和规矩

AGENTS.md 当目录来用->只写摘要和索引。AI需要什么就加载什么

2.执行能力：给AI装上手脚和工具

安装工具让AI可以操控文件读取代码，或者封装agentSkills技能包

3.任务编排：给AI安排好工作计划

大任务拆分成小人物，每次只做一个功能点

PlanMode：先让AI出方案，人工确认后再写代码

文档沉淀：每做完一个功能就记录当前实现、技术方案、待做事项

SubAgents并行：互不依赖的小任务同时进行，效率更高

4.反馈机制：让AI自己检查工作

跑Linter（代码检查工具）->语法和规范问题

跑自动化测试->验证功能是否正确

让AI自己打开浏览器实际操作一遍->功能可用才算完成

测试没通过->AI自动读取报错信息、分析原因、尝试修复

AI搞不定时，人工介入

5.架构护栏：防止代码越改越乱

AI会模仿仓库里已有的代码风格，哪怕是烂代码->同样的代码写好几遍、不知道拆组件->技术债越滚越大

1.架构linter->强制执行架构规则,->如UI层不能直接调用数据底层，模块间依赖必须单向

2.pre-commit Hooks->代码提交前自动拦截不合规的代码

3.垃圾回收->定期让AI扫描代码库，自动提交修复PR，持续偿还技术债

4.Git检查点->每完成一个功能就提交一次代码，万一改出问题就随时恢复

核心思想

Harness 不是完全放手不管->而是把人的精力都用在最关键的地方

5条实用Harness方法

1.做项目之前先写好AGENTS.md->项目背景、技术栈、代码规范

2.先让AI出方案并人工确认，再动手写代码

3.用MCP和Skills给AI配好工具->联网查资料、获取最新信息

4.做完功能一定要让AI自己跑测试验证->确保能正常运行

5.每完成一个功能就让AI沉淀文档并提交代码->作为存档点

Harness工具推荐

Spec Kit

SDD规范驱动开发->先拆解需求为详细规范文档->AI按文档一步步开发->每阶段有明确验收标准

Superpowers

Agent Skill框架->内置强制TDD(先写测试再写代码) 两阶段代码审查+子代理协作

总结

以前做项目->核心是写代码

现在AI能帮我们写越来越多的代码->得花更多的精力在需求分析、方案设计、任务拆解、质量把关
能不能用好AI，取决于你自己的工程能力有多强

原视频链接：[https://www.douyin.com/user/self?](https://www.douyin.com/user/self?from_tab_name=main&modal_id=7634196398245711142&showTab=like)

[from_tab_name=main&modal_id=7634196398245711142&showTab=like](https://www.douyin.com/user/self?from_tab_name=main&modal_id=7634196398245711142&showTab=like)